

COMPTE RENDU

CyberQuest

Projet VR | ANNÉE UNIVERSITAIRE : 2025–2026

Auteurs	Betchem Ngon Warren Shamir & Zaime Salma
Encadrant	Prof. Corentin Couprie
Établissement	ENSIM — Université du Mans
Date	16 janvier 2026

Table des matières

Résumé	2
1 Présentation du projet	3
1.1 Vision initiale du jeu	3
1.2 Évolution	3
2 KiAFéKoi (Répartition des tâches)	5
3 Difficultés rencontrées et solutions	6
4 Instructions (Mode d'emploi)	7
4.0.1 Mascotte et environnement	7
4.1 Déroulement du jeu et interactions	9
4.1.1 Exploration et compréhension des composants	10
4.1.2 Début de la mission	12
4.1.3 Manipulation des cubes	14
4.1.4 Fin de la mission	15

Résumé

En une phrase

CyberQuest est une expérience immersive en réalité virtuelle qui permet de **découvrir le fonctionnement interne d'un ordinateur** et d'**apprendre la réponse à incident** au travers d'interactions simples et signifiantes. (Durée cible : *5 minutes*.)

Objectifs pédagogiques

- Comprendre le rôle des composants matériels principaux (CPU, RAM, disque, carte réseau).
- Identifier les effets d'un virus informatique (corruption, fuite de données, désordre d'instructions).
- Réparer soi-même les dysfonctionnements via une remédiation guidée.

1. Présentation du projet

1.1 Vision initiale du jeu

Le joueur incarne un explorateur guidé par une **mascotte** (une puce électronique stylisée) dans une **ville métaphorique** qui représente la carte mère. Chaque site correspond à un composant essentiel (processeur, mémoire vive, disque, carte réseau). L'apprentissage se fait en réparant les pannes provoquées par une attaque simulée.

Un pont-réseau mène à la porte principale gardée par la **NIC** (Network Interface Card).

- **La Mascotte** : Petite, expressive et bienveillante, elle guide le joueur et commente les événements.
- **La RAM** : Imaginée comme un grand vestiaire ou une salle d'adresses hexadécimales où les instructions sont des cubes.
- **Le CPU** : Un four industriel ou une chaîne d'assemblage qui traite les instructions.
- **Le Driver** : Représenté comme une feuille ancienne servant de traducteur.

Le jeu devait se dérouler en 2 phases : un parcours guidé de 5 minutes pour découvrir les composants (flux verts normaux), suivi d'une alerte virale (lumières rouges). Le joueur devait alors répondre à l'incident en 15 minutes en réparant la RAM (replacer les cubes), le CPU (réordonner les instructions) et le réseau (corriger le driver).

1.2 Évolution

Au cours du développement du projet, plusieurs contraintes techniques et matérielles nous ont amenés à simplifier le scénario initial, tout en conservant l'objectif pédagogique du jeu. Le but principal est resté le même : permettre au joueur de comprendre le fonctionnement interne d'un ordinateur de manière intuitive et immersive.

Dans la version finale, le jeu se concentre principalement sur la mémoire vive (RAM) et la notion d'instructions/attaque "fuite de données".

Le joueur commence par une **courte phase d'introduction** qui présente l'univers et le contexte du jeu. Il dispose ensuite d'un temps limité pour observer librement la ville et identifier les différents composants.

Après cette phase de découverte, **une alerte sonore marque le début de la mission**. Le joueur doit alors réparer la mémoire en remplaçant correctement les instructions dans les bonnes cellules mémoire. Chaque instruction correspond à une action que le processeur

devra exécuter, et chaque emplacement représente une adresse mémoire précise.

Le processeur est toujours présenté comme le lieu où les instructions sont exécutées, tandis que le disque reste identifié comme un espace de stockage permanent. Ces éléments sont intégrés visuellement dans l'environnement afin de conserver la cohérence éducative du jeu.

Cette simplification permet de garantir une expérience fluide dans un temps de jeu court (**d'environ cinq minutes**) tout en maintenant une forte valeur pédagogique et une bonne immersion pour le joueur.

2. KiAFéKoi (Répartition des tâches)

Membre	Tâches principales et Rôles	Implication
Zaïme Salma	— Modélisation 3D de la mascotte et des composants principaux de la ville (RAM, CPU, Drive, éléments réseau, panneau d'entrée, cubes/instructions RAM) sous Blender. — Importation et intégration des assets dans Unity (placement de la scène, éclairages, éléments décoratifs). — Configuration complète du projet VR et tests sur casque. — Développement des scripts de gameplay et des interfaces (scénarios, interactions, logique de jeu). — Implémentation de la tâche mémoire avec gestion du chronomètre, affichage des panneaux de victoire et de défaite. — Mise à jour et simplification du scénario du jeu pour améliorer l'expérience utilisateur.	70%
Betchem Ngon Warren Shimir	— Proposition de l'idée globale du jeu et orientation conceptuelle. — Participation au design initial de la mascotte. — Création et configuration du dépôt Git (structure du projet, fichier <code>.gitignore</code>). — Recherche et sélection des assets graphiques et sonores. — Intégration des scripts audio et gestion de la musique dans le projet. — Génération des builds et Tests du build <i>XR device simulator</i>. — Rapport	30%

TABLE 2.1 – Tableau de répartition des tâches

3. Difficultés rencontrées et solutions

Problème 1 : Incompatibilité des versions de Blender et Unity

Description : Lors des premières phases de développement (fin novembre jusqu'à début décembre), **le projet a été réalisé sur des versions non conformes de Blender et Unity**. Cette incompatibilité a entraîné des problèmes d'importation, de stabilité et de compatibilité des scènes, rendant le travail initial de environ 2 semaines non réutilisable. Après identification du problème, il a été nécessaire de reprendre le projet depuis une base saine avec les versions correctes des outils. Cette situation a généré un risque de dépassement du délai.

Solution : Afin de garantir la livraison d'un produit fonctionnel, on a pris la décision de **simplifier le scénario du jeu en abandonnant des fonctionnalités initialement prévues** concernant la réparation du CPU et la gestion du flux réseau ont été abandonnées. Le projet conserve encore sa dimension pédagogique à travers la présentation des composants de la ville et une tâche principale centrée sur la réparation de la mémoire (RAM). Le temps de jeu a également été réduit par rapport à la version initiale (15 minutes).

Problème 2 : Limitation matérielle pour les tests VR et dysfonctionnement des interactions

Description : Lors des premiers tests effectifs sur casque, seules certaines interactions matérielles étaient fonctionnelles (déplacement via joystick et utilisation du trigger). Par contre, **les interfaces utilisateur et les boutons interactifs (navigation dans les écrans d'introduction, boutons *Next*, *Compris*, etc.) ne répondaient pas correctement** dans l'environnement VR, bien qu'ils fonctionnent correctement lors des tests en mode *Play* dans Unity. Plusieurs tentatives d'ajustement de la configuration des entrées ont été réalisées, mais sans parvenir à corriger totalement le problème dans le temps imparti.

Solution : Nous avons repensé les écrans d'introduction en supprimant toute interaction manuelle du joueur : la **navigation entre les écrans est désormais automatique** et basée sur des intervalles de temps courts. Ce qui permet d'éviter les blocages liés aux interactions VR tout en conservant une introduction claire et compréhensible pour le joueur.

4. Instructions (Mode d'emploi)

4.0.1 Mascotte et environnement

La mascotte du jeu est représentée sous la forme d'une puce graphique. Ce modèle n'a pas été importé depuis une bibliothèque externe. Elle apparaît uniquement à la fin du jeu afin d'annoncer le résultat au joueur et elle possède deux expressions principales :

- une expression **joyeuse (happy)** lorsque la mission est réussie ;
- une expression **anxieuse (anxious)** lorsque la mission échoue.

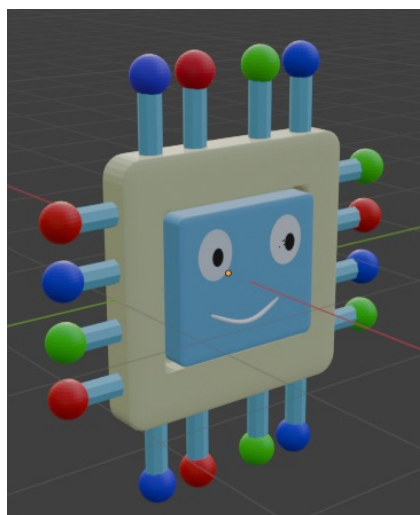


FIGURE 4.1 – Mascotte avec une expression joyeuse (succès de la mission).

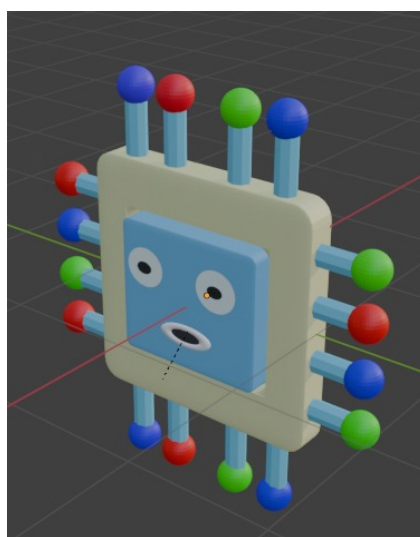


FIGURE 4.2 – Mascotte avec une expression anxieuse (échec de la mission).

L'environnement du jeu est une ville minimaliste conçue pour rester claire et facile à comprendre. Elle contient trois composants principaux :

- la **RAM**, représentant la mémoire de travail ;
- le **CPU**, représentant l'unité de calcul ;
- le **Drive**, représentant le stockage permanent.

-Une zone d'entrée est aussi présente pour symboliser l'arrivée du flux réseau, représenté visuellement par des objets de forme octaédrique.

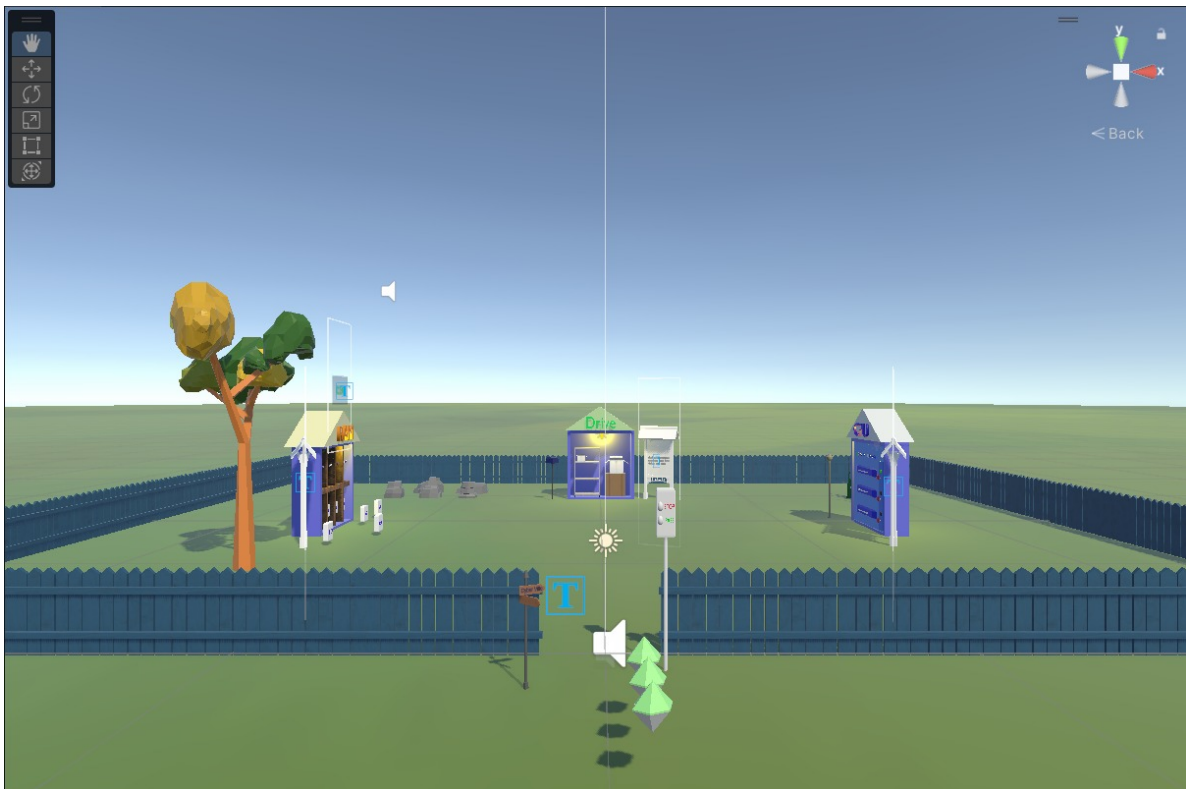


FIGURE 4.3 – Vue globale de la ville et des principaux composants du jeu.

4.1 Déroulement du jeu et interactions

-Le jeu commence par une série d'écrans introductifs. Ces interfaces présentent l'univers du jeu et expliquent au joueur ce qu'il va faire : bienvenue dans la ville, ce qui s'est produit, et l'objectif de la mission.

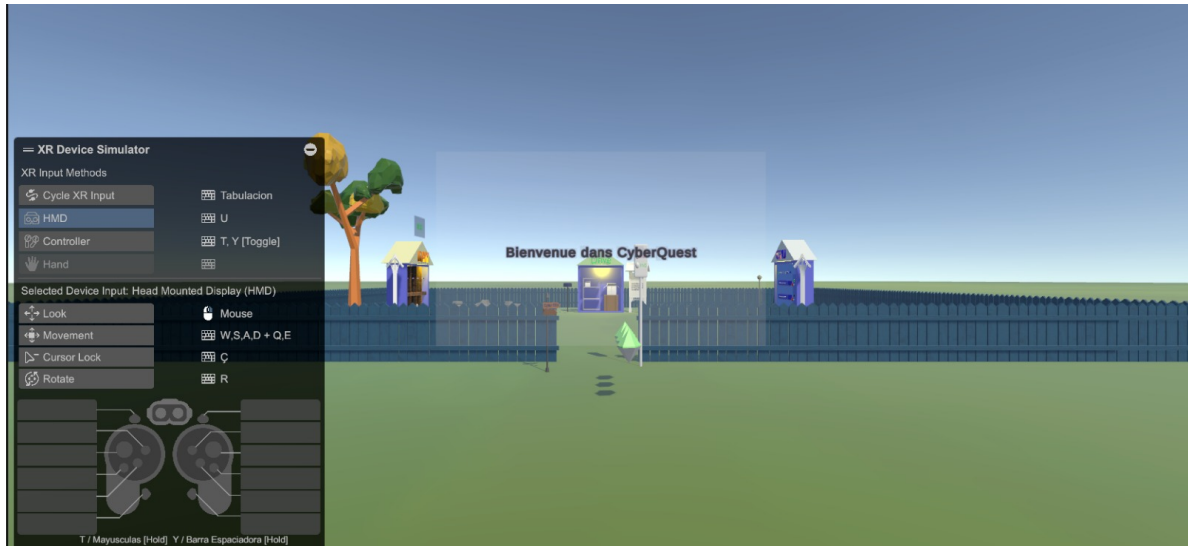


FIGURE 4.4 – Écran d'introduction présentant le jeu.

-Le joueur est ensuite invité à découvrir la ville afin de comprendre le rôle de chaque composant en lisant les panneaux pédagogiques placés à proximité.



FIGURE 4.5 – Annonce de la phase de découverte de la ville.

-Durant cette phase d'introduction, aucune interaction n'est possible. Le joueur ne peut ni se téléporter, ni manipuler des objets.

-Lorsque le dernier écran disparaît, le joueur récupère le contrôle et peut commencer à explorer la ville librement pendant environ 40 secondes.

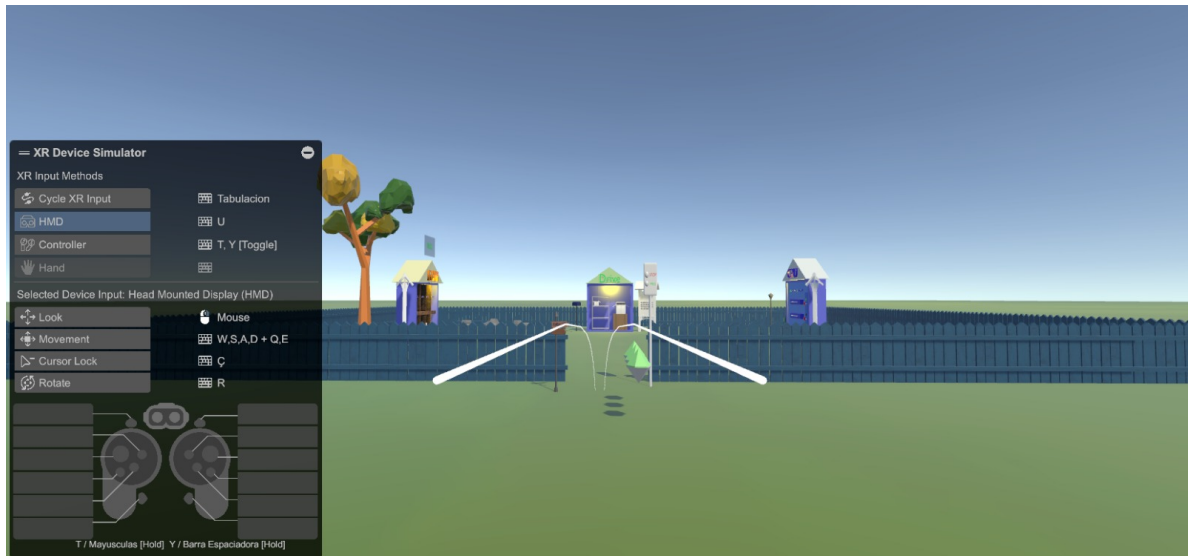


FIGURE 4.6 – Début de la phase de découverte de la ville.

4.1.1 Exploration et compréhension des composants

-Pendant l'exploration, le joueur observe les différents bâtiments représentant les composants informatiques et lit les panneaux explicatifs.

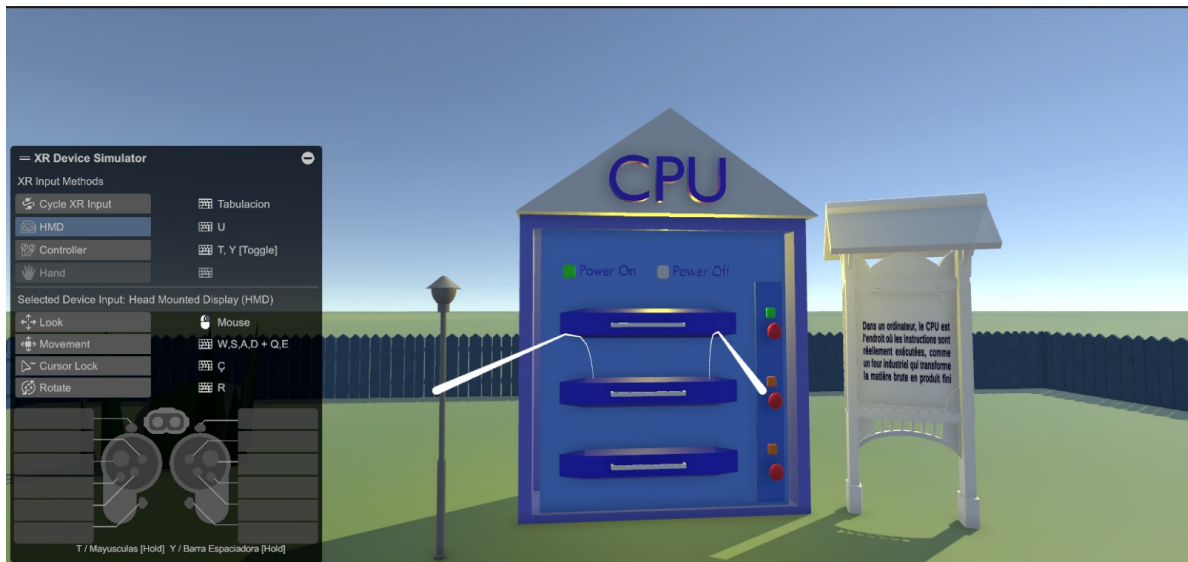


FIGURE 4.7 – Zone représentant le processeur (CPU) et son panneau explicatif.

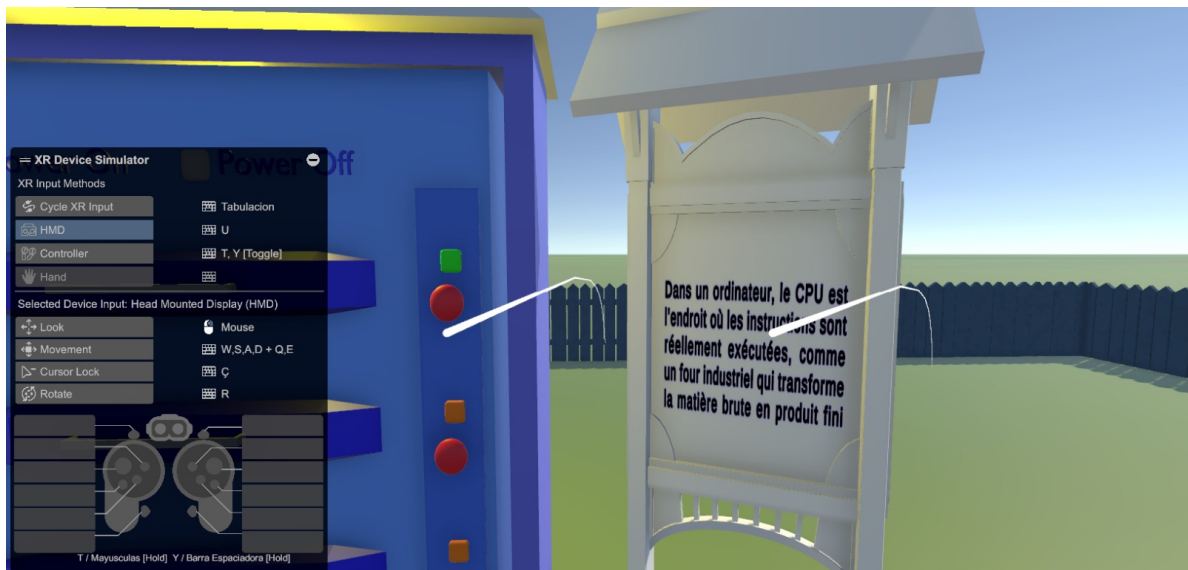


FIGURE 4.8 – Panneau explicatif de CPU.



FIGURE 4.9 – Zone représentant le disque de stockage et son panneau explicatif.



FIGURE 4.10 – Zone représentant la mémoire RAM.

4.1.2 Début de la mission

-À la fin des 40 secondes d’exploration, un signal sonore est émis. Ce bip annonce le début de la mission de réparation de la mémoire.

-Le joueur doit alors se téléporter le plus près possible de la zone RAM afin de commencer la tâche.

-Un chronomètre situé au-dessus de la RAM démarre automatiquement. Le joueur dispose d’une minute et trente secondes pour terminer la mission.

-La couleur du minuteur évolue progressivement pour indiquer l’urgence : vert au début, puis orange, et enfin rouge lorsque le temps touche à sa fin.



FIGURE 4.11 – Minuteur de la mission avec changement de couleur (*phase initiale*)



FIGURE 4.12 – Minuteur de la mission avec changement de couleur (*phase intermédiaire*)



FIGURE 4.13 – Minuteur de la mission avec changement de couleur (*phase finale : countdown*)

4.1.3 Manipulation des cubes

-Les cubes représentent les instructions à remettre dans les bonnes cellules mémoire.

-Pour saisir un cube, le joueur le pointe avec le rayon du contrôleur et appuie sur le **trigger**. Le cube est alors attrapé (**IL FAUT MAINTENIR LE TRIGGER sans le relâcher.**)

-Pour placer le cube, le joueur utilise le **joystick** afin de l'avancer jusqu'au fond de la case. Une légère aimantation permet au cube de se positionner automatiquement dans la cellule.

-Si le cube est placé dans la bonne case, la face centrale s'allume en vert et un son positif est joué.



FIGURE 4.14 – Cube mis dans la bonne une cellule mémoire.

-Dans le cas contraire, elle s'allume en rouge et un son d'erreur est entendu.



FIGURE 4.15 – Cube mis dans la mauvaise une cellule mémoire.

-Le joueur peut retirer un cube et corriger son emplacement à tout moment.

4.1.4 Fin de la mission

-Si toutes les instructions sont correctement remplacées avant la fin du temps imparti, la mascotte apparaît avec une expression joyeuse et remercie le joueur.

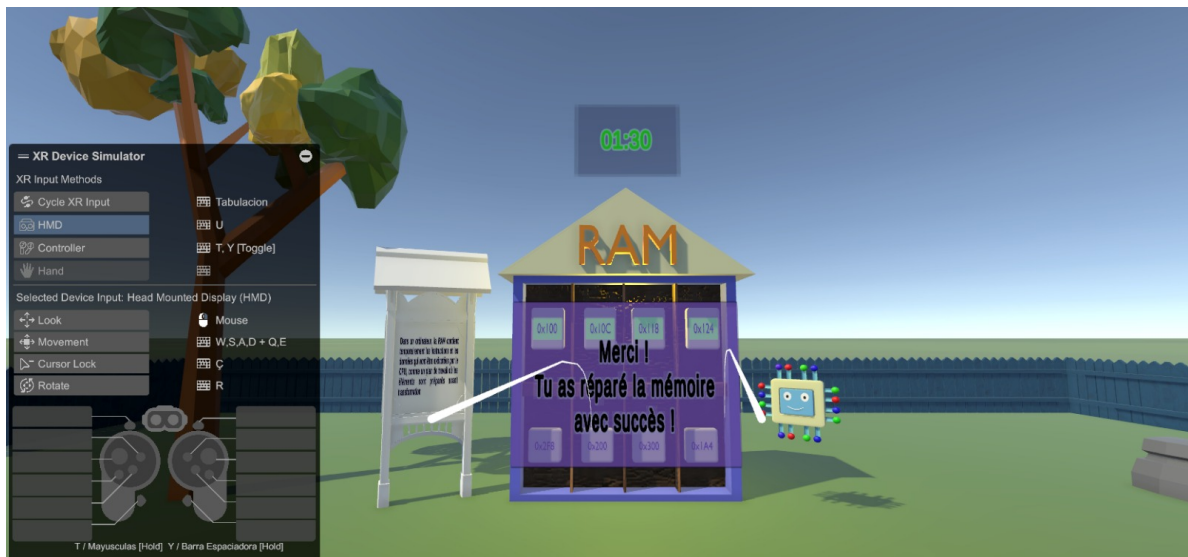


FIGURE 4.16 – Message de réussite et mascotte satisfaite.

-Si le temps est écoulé avant la fin de la réparation, la mascotte apparaît avec une expression anxieuse et un message d'échec est affiché.



FIGURE 4.17 – Échec de la mission et affichage du message Hard Luck.